

Série C : Exceptions

Table des matières

Série C : Exceptions.....	1
Exercice C.1: Extraits du tutoriel Java de Oracle.....	1
Exercice C.2: Apprendre à connaître les exceptions.....	3
Exercice C.3: FractionList I.....	4
Exercice C.4: FractionList II.....	4
Exercice C.5: Crypter.....	4

Exercice C.1: Extraits du tutoriel Java de Oracle

Question 1:

Is the following code legal?

```
try {  
    ...  
} finally {  
    ...  
}
```

Answer:

[Yes, it's legal — and very useful](#) A try statement does not have to have a catch block if it has a finally block. If the code in the try statement has multiple exit points and no associated catch clauses, the code in the finally block is executed no matter how the try block is exited. Thus it makes sense to provide a finally block whenever there is code that must always be executed. This include resource recovery code, such as the code to close I/O streams.

Question 2:

What exception types can be caught by the following handler?

```
catch (Exception e) {  
  
}
```

What is wrong with using this type of exception handler?

Answer:

[This handler catches exceptions of type Exception; therefore, it catches any exception. This can be a poor implementation because you are losing valuable information about the type of exception being thrown and making your code less efficient. As a result, your program may be forced to determine the type of exception before it can decide on the best recovery strategy.](#)

Question 3:

Is there anything wrong with this exception handler as written? Will this code compile?

```
try {  
  
    } catch (Exception e) {  
  
    } catch (ArithmeticException a) {  
  
    }
```

Answer:

This first handler catches exceptions of type Exception; therefore, it catches any exception, including ArithmeticException. The second handler could never be reached. This code will not compile.

Question 4:

Match each situation in the first list with an item in the second list.

- a) `ArrayList<String> sl; String s = sl.get(0) ;`
 - b) The JVM starts running your program, but the JVM can't find the Java platform classes. (The Java platform classes reside in `classes.zip` or `rt.jar`.)
 - c) A program is reading a stream and reaches the end of stream marker.
 - d) Before closing the stream and after reaching the end of stream marker, a program tries to read the stream again.
- 1. error
 - 2. checked exception
 - 3. compile error
 - 4. no exception

Answer:

a → 3 (compile error). The ArrayList is not initialized and will not compile.

b → 1 (error).

c → 4 (no exception). When you read a stream, you expect there to be an end of stream marker. You should use exceptions to catch unexpected behavior in your program.

d → 2 (checked exception).

In general, checked exceptions represent errors outside the control of the program. For example, the constructor of `FileInputStream` throws `FileNotFoundException` if the input file does not exist.

Exercice C.2: Apprendre à connaître les exceptions

Expérimentez avec les exceptions, et notez chaque fois vos constatations, les messages obtenus et vos conclusions :

- Créez une classe avec une méthode lançant une exception obligatoire (checked exception), p.ex. une `IOException`.

```
public void criticalMethod() throws IOException {
    BufferedReader inputStream =
        new BufferedReader( new FileReader("IDoNotExist.txt")); //=> try...
    inputStream.readLine();
}
ou bien
public void criticalMethod() throws IOException {
    throw new IOException("Hi, I am an exception!");
}
```

- Traitez les exceptions uniquement par `throws` sans utiliser `try catch`.
- Appelez la méthode critique par une autre méthode dans la même classe

```
public void callCriticalMethod() throws IOException {
    criticalMethod();
}
```

→ Message (avant `throws IOException`) : `CriticalClass.java @ line 17: unreported exception java.io.FileNotFoundException; must be caught or declared to be thrown`

- Appelez la méthode critique par une autre méthode dans une autre classe.

```
import java.io.IOException;
public class CriticalClassCaller
{
    public void callCriticalMethod() throws IOException {
        new CriticalClass().criticalMethod();
    }
}
```

- Ajoutez une troisième classe appelant la méthode critique de la deuxième classe. Faites l'appel dans la méthode `main(...)`.

```
import java.io.IOException;
public class CriticalClassCallerLevel2
{
    public static void main(String args[]) throws IOException {
        new CriticalClassCaller().callCriticalMethod();
    }
}
```

- Que remarquez-vous si la classe 'supérieure' (dans la hiérarchie des appels) ne traite pas l'exception, mais effectue aussi un `throws` ?
- Traitez les exceptions par `try catch` aux différents niveaux. Enlevez les instruction `throws` superflues. Faites afficher un '**stack trace**' dans le cas d'une erreur.

```
public void criticalMethod() /* plus rien */ {
    try {
        BufferedReader inputStream =
            new BufferedReader( new FileReader("IDoNotExist.txt"));
        inputStream.readLine();
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}
```

- Refaites le même exercice avec une exception du type `RuntimeException`. (p.ex. `ArithmeticException`).

La clause **throws** n'est plus nécessaire. L'exception est lancée et elle peut être traitée à tous les niveaux. Mais attention aux instructions derrière les instr.critiques... → voir point suivant :

- Entourez l'instruction critique et les appels des méthodes critiques par des affichages p.ex : `"callCriticalMethod2: before call"`, `"callCriticalMethod2: after call"`. Ajoutez un affichage à la fin des méthodes. Faites des essais en plaçant `try-catch` aux différents niveaux (dans les différentes méthodes). Que constatez-vous ?

Sans `try-catch` aucune instruction n'est effectuée dans les méthodes critiques après l'appel critique.

```
main: before call
callCriticalMethod2: before call
callCriticalMethod: before call
criticalMethod: before division
ERROR: Sourced file: inline evaluation of: ``CriticalClassCa
of: ``CriticalClassCaller2.main(null);`` : CriticalClassCall
Target exception: java.lang.ArithmeticException: / by zero
```

Si le bloc `try-catch` est ajouté à la méthode extérieure (`main`) alors les instructions derrière les appels critiques dans les méthodes intérieures ne sont pas exécutées. Dans `main`, le bloc derrière le bloc `try-catch` est exécuté correctement.

```
main: before call
callCriticalMethod2: before call
callCriticalMethod: before call
criticalMethod: before division
java.lang.ArithmeticException: / by zero
main: end
```

Si le bloc `try-catch` est ajouté à la méthode intérieure alors les instructions derrière les appels critiques dans toutes les méthodes appelantes sont exécutées. (Seules les instructions suivant à l'instruction critique dans le bloc `try catch` de la méthode intérieure sont pas exécutées.)

```
main: before call
callCriticalMethod2: before call
callCriticalMethod: before call
criticalMethod: before division
java.lang.ArithmeticException: / by zero
criticalMethod: end
callCriticalMethod: after call
callCriticalMethod: end
callCriticalMethod2: after call
callCriticalMethod2: end
main: end
```

En conclusion :

L'exécution se poursuit normalement à partir du bloc `catch` ayant traité l'exception survenue. Les méthodes appelantes sont exécutées complètement, comme s'il n'y avait pas eu d'erreur. On ne revient pas aux méthodes appelées (qui ont négligé l'erreur). La clause `throws` n'y change rien.

- Ajoutez des blocs `try ... finally` à toutes les méthodes, désactivez les

blocs **catch** et observez la suite des traitements.

Sans blocs **catch**, tous les blocs **finally** sont exécutés avant de lancer l'exception. Les blocs entre l'instruction critique et **finally** ainsi que derrière le bloc **finally** NE SONT PAS EXECUTES !

Avec les blocs **catch**, l'exécution des autres blocs (derrière les appels critiques, ou derrière les blocs **try...catch...finally**) fonctionne de la même façon qu'avant.

- Est-ce que les deux blocs suivants réagissent de la même façon ? Motivez votre réponse !

```
(A)    try {  
        <bloc d'instructions 1>  
    }  
    catch (...) {  
        <bloc d'instructions 2>  
    }  
    finally {  
        <bloc d'instructions 3>  
    }
```

```
(B)    try {  
        <bloc d'instructions 1>  
    }  
    catch (...) {  
        <bloc d'instructions 2>  
    }  
    <bloc d'instructions 3>
```

Non, les deux blocs ne réagissent pas de la même façon. Si le <bloc d'instructions 1> provoque une exception qui n'est pas traitée par le bloc **catch**, alors sans **finally** le <bloc d'instructions 3> n'est jamais exécuté.

Ceci peut être le cas, si une autre exception a été provoquée que celle qui est prévue dans le bloc **catch**. ou aussi, si le bloc **catch** lance une autre exception (avec **throw**) en réponse à l'exception interceptée.

Exercice C.3: FractionList I

Même si dans le cours, nous avons défini une exception du type **DivisionByZeroException**, pour le cas où le dénominateur est zéro, ce n'est pas la meilleure manière de procéder, car dans ce cas c'est trop tard : la fraction a déjà été initialisée avec la valeur indésirable.

Mieux vaut éviter ce cas dès le départ :

1. Reprenez votre classe **Fraction** et lancez une exception du type **IllegalArgumentException** dès qu'on essaie d'affecter une valeur nulle au dénominateur. Enlevez tous les autres tests sur une valeur nulle du dénominateur. Affichez un message d'erreur informatif.
2. Testez l'exception, en Unimozer.
3. Développez un programme (en Unimozer ou en NetBeans) pour tester votre classe. Interceptez l'erreur et affichez son message à l'écran.
4. Après avoir ajouté une méthode **divideBy(Fraction)** à votre classe, un autre cas de figure se présente : Définissez un nouveau type d'exception **DivisionByZeroException** (dérivé de **ArithmeticException**) que vous provoquez dans ce cas.
5. Testez les deux exceptions, en Unimozer, puis développez une application pour tester toutes les opérations et les exceptions qu'elles peuvent provoquer. Interceptez les deux exceptions avec deux blocs **try ... catch**.

Exercice C.4: FractionList II

Copiez l'exercice précédent et élargissez la copie comme suit :

L'application **FractionList** sait produire au hasard des fractions avec des valeurs aléatoires positives ou négatives (p.ex. entre -5 et 5) comme numérateur et comme dénominateur. La liste des fractions est à représenter à l'écran.

Si le générateur de nombres aléatoires produit un dénominateur zéro, alors la fraction n'est pas produite et elle n'est pas ajoutée à la liste. Utilisez **try-catch(-finally)** et pas de structure **if**. A la fin, le nombre de fractions dans la liste doit correspondre au nombre demandé.

Affichez aussi le nombre de fractions incorrectes survenues.

Exercice C.5: Crypter

Reprenez la classe **Crypter**:

1. Déterminez les cas où il peut s'avérer nécessaire de produire une exception. Testez votre classe pour ce cas. Quel type d'exception est approprié ?
2. Définissez un type d'exception individualisé pour ce type d'erreur.
3. Testez les exceptions avec une classe qui intercepte l'exception et réagit en conséquence.
4. Définissez un message par défaut pour le nouveau type d'exception.